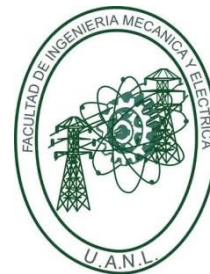




Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica



Controladores y Microcontroladores
Programables

Actividad Fundamental N.º04 –
Análisis de la arquitectura interna de un
microcontrolador

N.º de Equipo: 4

Brigada: 001

Nombre:	Matricula:	Numero de lista:
Heidi Pamela Martínez Martínez	1952947	6
Steven Neftalí Verdugo Figueroa	2132366	60
Ángel Armando Solís Jiménez	2132088	55
Carlos Emiliano Mendoza Ramírez	2048186	35

Nombre del profesor: Jesús Daniel Garza Camarena

Semestre Agosto – Diciembre 2025

Días de la clase y Hora: JueM4M6

San Nicolás de los Garza, N.L.

Fecha: 18/09/2025

COMPETENCIA ESPECÍFICA

Desarrollar habilidades en la programación y manejo de los puertos de entrada y salida de un microcontrolador. Este proyecto integrará conocimientos sobre tablas de verdad, manipulación de bits y control de hardware a nivel bajo, asegurando que los estudiantes apliquen de manera efectiva los conceptos adquiridos durante el curso.

En esta actividad práctica, los estudiantes diseñarán y programarán un sistema de control de secuencias de luces utilizando un microcontrolador. El sistema debe responder a diferentes combinaciones de dos botones de entrada y controlar 6 LEDs en la salida junto con un display 7 segmentos sin un decodificador con las siguientes secuencias:

BTN1	BTN0	LEDS x 6	Display 7 segmentos
0	0	Efecto "Auto Fantástico" con una velocidad de 0.5 segundos	Muestra número 0
0	1	Todos los LEDs apagados	Muestra número 1
1	0	Secuencia de LEDs libre, definida por el estudiante	Muestra número 2
1	1	Todos los LEDs encendidos	Muestra número 3

Nota: Utiliza macros, etiquetas y funciones para mejorar la calidad y la claridad de tu código. La transición entre funciones no tiene que ser inmediata; sin embargo, investiga cómo implementarla de manera efectiva.

INTRODUCCIÓN

El estudio de los microcontroladores se ha convertido en un pilar fundamental dentro del campo de la electrónica y los sistemas embebidos, ya que constituyen la base de gran parte de la tecnología que utilizamos cotidianamente, desde electrodomésticos y sistemas de iluminación, hasta dispositivos médicos y automotrices. Comprender su arquitectura interna y su funcionamiento permite no solo adquirir habilidades técnicas en la programación y manejo de puertos de entrada y salida, sino también desarrollar un pensamiento lógico y estructurado para resolver problemas mediante soluciones electrónicas eficientes.

Dentro de este proceso de aprendizaje, las tablas de verdad desempeñan un papel crucial. Estas representan de manera ordenada todas las posibles combinaciones de entradas y sus salidas correspondientes, lo cual facilita la interpretación y la correcta implementación de sistemas digitales. En la programación de microcontroladores, las tablas de verdad permiten traducir condiciones lógicas abstractas en instrucciones claras y ejecutables, logrando así el control preciso de dispositivos electrónicos como LEDs, botones y pantallas. Gracias a ellas, los estudiantes pueden visualizar cómo un cambio en una entrada se refleja directamente en la salida, reforzando la relación entre teoría y práctica.

Asimismo, esta actividad fomenta el análisis comparativo de diferentes tecnologías como los microprocesadores, los microcontroladores, las FPGA y los ASIC, entendiendo sus diferencias en términos de estructura, flexibilidad, costo y velocidad. De igual manera, se profundiza en la importancia de los lenguajes de programación como C y Assembly, que son herramientas indispensables para interactuar con el hardware a bajo nivel. El uso de macros, etiquetas y funciones no solo incrementa la claridad y legibilidad del código, sino que también fortalece la disciplina en la escritura de programas más eficientes y organizados. El desarrollo de un sistema de control de secuencias de luces con un microcontrolador no solo constituye un ejercicio práctico de programación, sino también una experiencia formativa integral. El estudiante aprende a integrar conocimientos de lógica digital, manipulación de bits y control de periféricos, al mismo tiempo que fortalece su capacidad de diseñar e implementar proyectos electrónicos reales. De esta forma, la actividad contribuye a consolidar competencias esenciales en la ingeniería, sentando las bases para enfrentar retos más complejos en el diseño de sistemas embebidos.

Análisis de la arquitectura interna de un microcontrolador

1. INVESTIGACIÓN Y DEFINICIONES

En esta sección se presentan definiciones detalladas de conceptos fundamentales en la electrónica y programación de sistemas embebidos. Las fuentes de consulta incluyen libros académicos, blogs especializados y material educativo de confianza, garantizando así un contenido riguroso y bien fundamentado.

- **MICROPROCESADOR**

Un microprocesador es el circuito integrado central de un sistema de cómputo que ejecuta instrucciones almacenadas en memoria. Su función principal es procesar datos mediante operaciones aritméticas, lógicas y de control. Se encuentra comúnmente en computadoras personales y servidores, y su diseño está optimizado para alcanzar altas velocidades de procesamiento.

- **MICROCONTROLADOR**

Un microcontrolador es un circuito integrado que combina en un solo chip un procesador, memoria y periféricos de entrada/salida. Está diseñado para aplicaciones embebidas que requieren control dedicado de dispositivos, como electrodomésticos, sistemas automotrices y robots. Su enfoque es la integración y bajo consumo de energía.

- **FPGA**

Un FPGA (Field-Programmable Gate Array) es un dispositivo digital reconfigurable que permite programar su estructura lógica para implementar distintos circuitos. Ofrece gran flexibilidad y se utiliza en prototipado rápido, procesamiento de señales y aplicaciones donde el paralelismo es crítico.

- **ASIC**

Un ASIC (Application-Specific Integrated Circuit) es un chip diseñado específicamente para una aplicación concreta. A diferencia de los FPGAs, no son reconfigurables, pero alcanzan mayor eficiencia, menor consumo de energía y velocidad optimizada. Son comunes en teléfonos móviles, consolas de videojuegos y equipos de telecomunicaciones.

- **LENGUAJE ENSAMBLADOR (ASSEMBLY)**

El lenguaje ensamblador es un lenguaje de bajo nivel que se relaciona directamente con las instrucciones de la máquina. Cada instrucción en ensamblador corresponde casi de manera directa a una instrucción en código máquina, lo que permite un control detallado del hardware, aunque con mayor complejidad para el programador.

○ LENGUAJE C

El lenguaje C es un lenguaje de programación de propósito general ampliamente usado en sistemas embebidos debido a su eficiencia, control de memoria y cercanía al hardware, sin perder legibilidad. C es considerado el lenguaje estándar para el desarrollo de firmware y aplicaciones críticas en sistemas electrónicos.

2. COMPARACIÓN TÉCNICA

A continuación, se presenta una tabla comparativa detallada entre FPGA, Microcontroladores, ASIC y Microprocesadores. Se destacan aspectos clave como su estructura, usos, flexibilidad, velocidad y costo.

Dispositivo	Estructura	Uso típico	Flexibilidad	Velocidad	Costo
Microprocesador	Unidad central de procesamiento de propósito general.	PCs, laptops, y servidores.	Baja, solo puede ejecutar software predefinido.	Alta, optimizado para múltiples tareas.	Alto costo relativo.
Microcontrolador	Incluye CPU, memoria y periféricos en un solo chip.	Sistemas embebidos, electrodomésticos, controladores.	Media, depende del hardware incorporado.	Moderada, adecuada para tareas específicas.	Muy bajo costo.
FPGA	Matriz de compuertas programables reconfigurables.	Prototipado, procesamiento paralelo, investigación.	Muy alta, puede reprogramarse múltiples veces.	Muy alta en tareas paralelas.	Costo elevado en comparación con microcontroladores.
ASIC	Chip diseñado para una aplicación específica.	Teléfonos, consolas, telecomunicaciones.	Nula, no puede reprogramarse.	Muy alta, optimizada para la tarea.	Alto costo de diseño, bajo costo en producción masiva.

3. DIFERENCIAS ENTRE LENGUAJES DE PROGRAMACIÓN

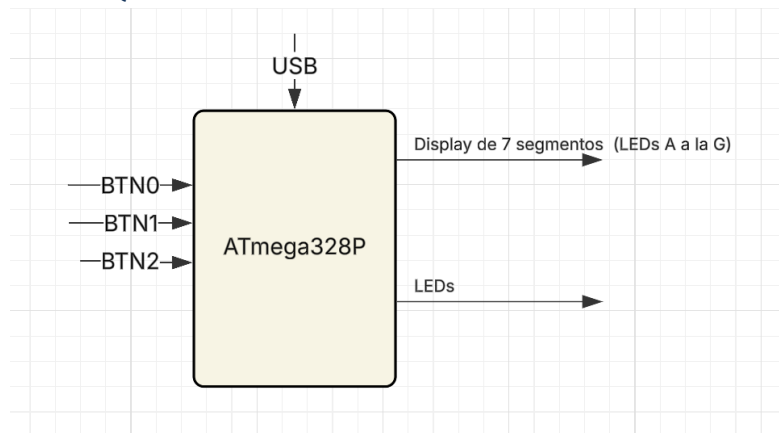
Criterio	Lenguaje C	Lenguaje Assembly (ASM)
Nivel de abstracción	Es un lenguaje de alto nivel, aunque muy cercano al hardware. Permite escribir programas más legibles y fáciles de mantener. El compilador se encarga de traducir el código a instrucciones de máquina.	Es un lenguaje de bajo nivel, casi una representación directa del código máquina. Cada instrucción ASM corresponde casi uno a uno a una instrucción del procesador. Requiere un conocimiento profundo de la arquitectura del microcontrolador.
Eficiencia	El compilador optimiza el código, pero no siempre al máximo nivel posible. El rendimiento depende en gran parte de la calidad del compilador. Generalmente suficiente para la mayoría de las aplicaciones de microcontroladores.	Permite un control total sobre cada instrucción. El programador puede optimizar al máximo en cuanto a velocidad y uso de memoria. Muy útil en rutinas críticas (por ejemplo, interrupciones de tiempo real o control de hardware muy fino).
Control del hardware	Ofrece una buena abstracción, pero se depende de librerías y del compilador para interactuar con registros y periféricos. Aunque se puede acceder a registros de bajo nivel, la sintaxis no es tan directa como en ASM.	Control absoluto sobre el hardware. Se puede manejar directamente registros, flags y periféricos sin capas intermedias. Ideal cuando se necesita precisión a nivel de ciclos de reloj.
Complejidad	Más fácil de aprender, escribir y mantener. Favorece la portabilidad: el mismo código puede adaptarse a diferentes microcontroladores con pocos cambios.	Más complejo y difícil de aprender. El código suele ser más largo, difícil de depurar y mantener. Poco portable: un programa escrito en ASM para un microcontrolador no funciona en otro con distinta arquitectura.

	Facilita proyectos grandes gracias a estructuras, funciones y modularidad.	
Ventajas	Código más claro y mantenible. Buena portabilidad entre plataformas. Permite desarrollo rápido y organizado. Adecuado para aplicaciones medianas y grandes.	Máximo control del hardware. Posibilidad de optimizar el rendimiento y el uso de memoria al detalle. Ideal para rutinas críticas de tiempo real.
Desventajas	Menor control a nivel de ciclos de reloj e instrucciones. Dependencia del compilador para la eficiencia. Puede generar código más grande o lento en comparación con ASM.	Curva de aprendizaje más difícil. Código largo y difícil de mantener. Poco portable entre diferentes microcontroladores. Desarrollo más lento y propenso a errores.

4. LISTA DE MATERIALES

Material	Marca	Precio (\$)	Unidades
Arduino R3	STEREN	180	1
Protoboard	STEREN	120	1
Resistencia	STEREN	10	14
Led	STEREN	3	6
Jumpers	STEREN	10	18
Display 7 segmentos	STEREN	15	1

5. DIAGRAMA DE BLOQUES



8. CÓDIGO EN LENGUAJE C O ASM NO ARDUINO

```

/*****
 *   Copyright (C) 2025 by Equipo 4
 *   Heidi Pamela Martínez Martínez          1952947   #6
 *   Steven Neftalí Verdugo Figueroa         2132366   #60
 *   Ángel Armando Solís Jiménez             2132088   #55
 *   Carlos Emiliano Mendoza Ramírez         2048186   #35
 *
 *   Attribution-NonCommercial-ShareAlike 3.0(CC-BY-NC-SA 3.0)
 *   AFU04 - Arquitectura Internas
 *   Dispositivo: atmega328p
 *   Lenguaje C
 *   Rev: 1.0
 *   Licencia:
 *
 *                                           Fecha: 18/09/25
 *****/

```

```

#define F_CPU 16000000UL // 16 MHz

#include <avr/io.h>
#include <util/delay.h>

// Macros para los botones
#define BTN0 (!(PINC & (1<<PC2)))
#define BTN1 (!(PINC & (1<<PC3)))
#define BTN2 (!(PINC & (1<<PC6)))

// LEDs en PD2-PD7
#define LED_MASK 0xFC

// Números para display 7 segmentos (anodo común - LOW enciende)
// Orden: a,b,c,d,e,f,g
uint8_t numeros[4][7] = {
    {0,0,0,0,0,0,1}, // 0
    {1,0,0,1,1,1,1}, // 1
    {0,0,1,0,0,1,0}, // 2
    {0,0,0,0,1,1,0}  // 3
};

// Declarar funciones
void mostrarNumero(uint8_t n);
void efectoAutoFantastico(void);
void secuenciaLibre(void);
void todosEncendidos(void);
void todosApagados(void);

int main(void) {
    // Configuración
    DDRB = 0x3F; // PB0-PB5 como salida (segmentos a-f)
    DDRC |= (1<<PC0); // PC0 como salida (segmento g)
    DDRC &= ~(1<<PC2)|(1<<PC3)|(1<<PC6); // botones entrada
    PORTC |= (1<<PC2)|(1<<PC3)|(1<<PC6); // pull-ups
    DDRD |= LED_MASK; // LEDs como salida

    while(1){

```

```

uint8_t btn0 = BTN0 ? 1 : 0;
uint8_t btn1 = BTN1 ? 1 : 0;
uint8_t btn2 = BTN2 ? 1 : 0;

if(btn1==0 && btn0==0){
    mostrarNumero(0);
    efectoAutoFantastico();
}
if(btn1==0 && btn0==1){
    mostrarNumero(1);
    todosApagados();
}
if(btn1==1 && btn0==0){
    mostrarNumero(2);
    secuencialibre();
}
if(btn1==1 && btn0==1){
    mostrarNumero(3);
    todosEncendidos();
}
// Este se mantiene al final
if(btn2==1){
    todosApagados();
    PORTB = 0x3F;      // apaga segmentos a-f
    PORTC |= (1<<PC0); // apaga g
}
}

// Mostrar número en display de anodo comun
void mostrarNumero(uint8_t n) {
    if(n>3) return;

    // Apagar todos primero (todos HIGH en anodo común)
    PORTB = 0x3F;
    PORTC |= (1<<PC0);

    // Encender segmentos necesarios (colocando LOW)
    for(uint8_t i=0; i<6; i++){
        if(numeros[n][i] == 0) // 0 = encendido
            PORTB &= ~(1<<i);
    }
    if(numeros[n][6] == 0) // segmento g
        PORTC &= ~(1<<PC0);
}

// Secuencia Auto Fantastico
void efectoAutoFantastico(void){
    static uint8_t dir = 1; // direccion
    static uint8_t pos = 2; // LED actual

    PORTD = (1<<pos);
    _delay_ms(500);

    if(dir){
        pos++;
        if(pos>7) { pos=6; dir=0; }
    }
}

```

```

    } else {
        pos--;
        if(pos<2) { pos=3; dir=1; }
    }
}

// Secuencia libre (3 y 3)
void secuenciaLibre(void){
    static uint8_t state = 0;
    if(state == 0){
        PORTD = (1<<PD2) | (1<<PD3) | (1<<PD4);
        state = 1;
    } else {
        PORTD = (1<<PD5) | (1<<PD6) | (1<<PD7);
        state = 0;
    }
    _delay_ms(500);
}

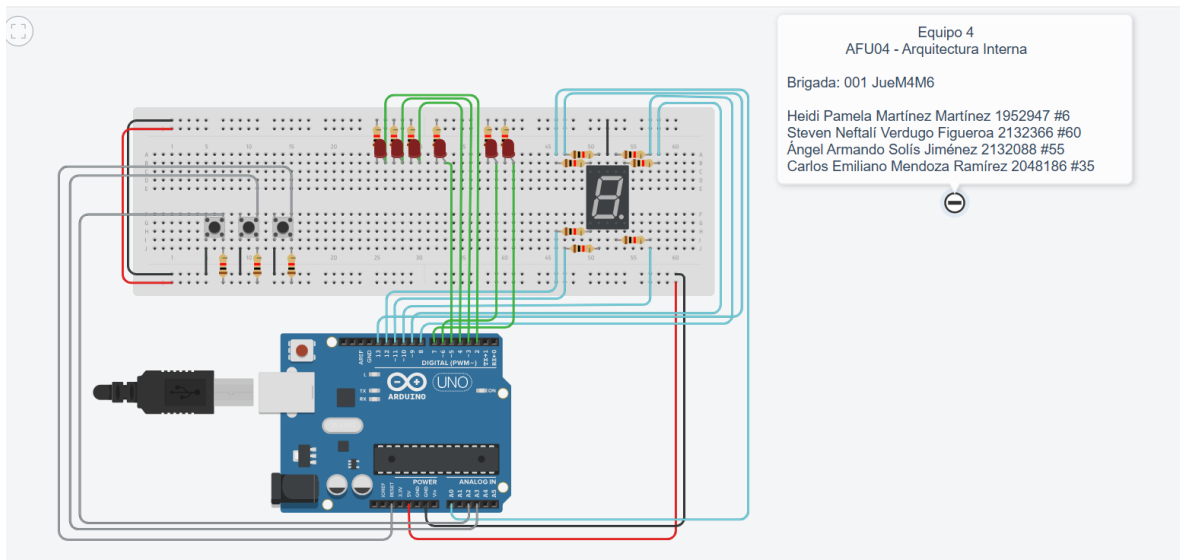
// Todos LEDs encendidos
void todosEncendidos(void){
    PORTD |= LED_MASK;
}

// Todos LEDs apagados
void todosApagados(void){
    PORTD &= ~LED_MASK;
}

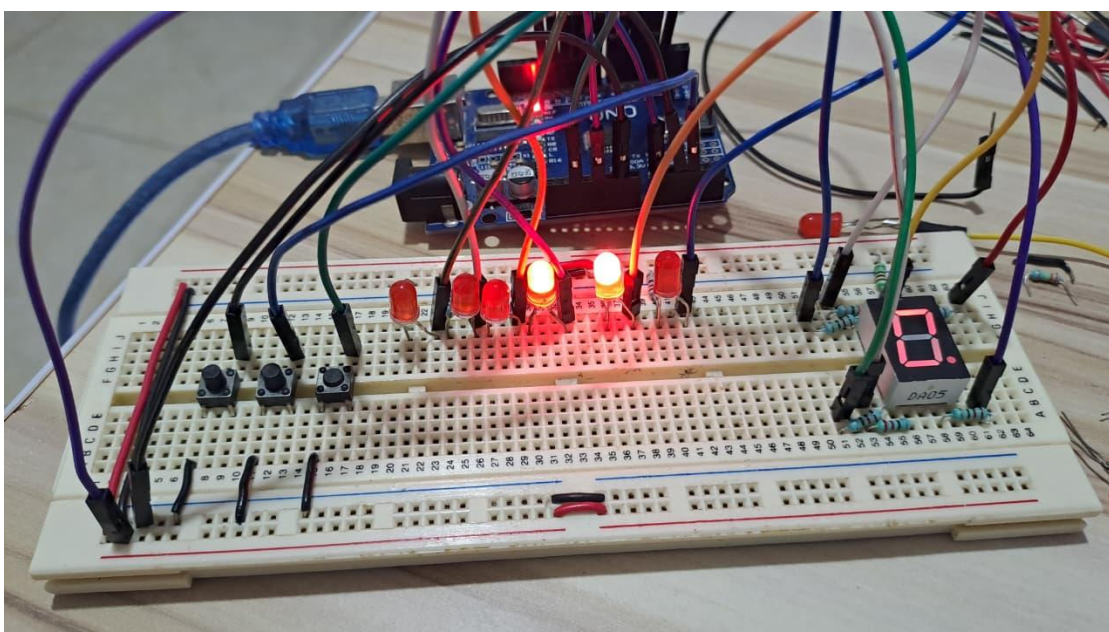
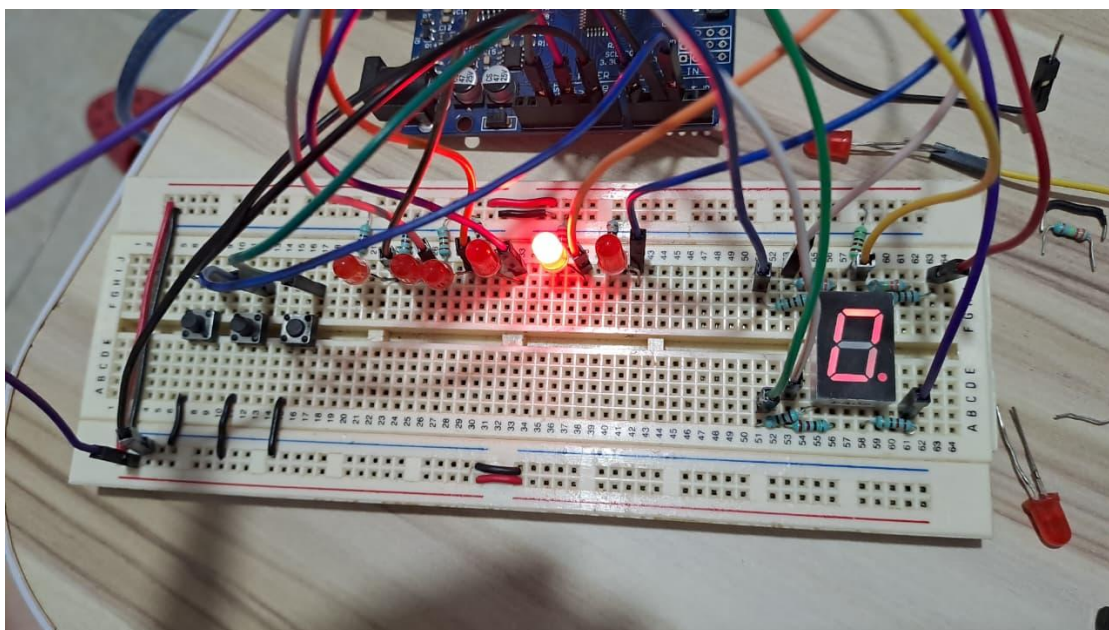
```

9. FOTOGRAFÍA DEL PROTOTIPO

ESQUEMÁTICO EN TINKERCAD



PROTOTIPO EN FÍSICO



CONCLUSIÓN PERSONAL

Heidi Pamela Martínez Martínez 1952947 #6

Durante la actividad aprendí a relacionar de manera práctica los conocimientos teóricos que he visto en clase sobre microcontroladores, puertos de entrada y salida, así como la lógica de control en C. Uno de los principales desafíos de esto fue el retomar los conocimientos de Proteus que ya tenía, pero con los archivos de la clase fue fácil volver a tomar el rumbo. Además, tuve que resolver problemas relacionados con el funcionamiento del display de 7 segmentos y la correcta detección de los botones en el código, pero esta experiencia me permitió reforzar mi capacidad de depuración y mi paciencia al enfrentar errores de hardware y software, ya que muchas veces el problema no estaba en la programación, sino en las conexiones físicas o en la forma de plantear las condiciones. En conclusión, siento que fue un aprendizaje completo, porque no solo reforcé mis conocimientos técnicos, sino también la mejor manera de resolución.

Steven Neftalí Verdugo Figueroa 2132366 #60

Esta actividad me permitió comprender de manera más clara el papel que juegan los microcontroladores en el desarrollo de sistemas electrónicos y cómo, a través de ellos, es posible controlar dispositivos de forma eficiente. Al trabajar con las tablas de verdad y programar las secuencias de LEDs, pude comprobar de manera práctica la relación entre la lógica digital y el comportamiento real del hardware, lo que facilitó mucho mi aprendizaje. Además, la comparación entre diferentes tecnologías como microprocesadores, FPGA y ASIC, así como entre los lenguajes C y Assembly, me ayudó a valorar las ventajas y desventajas de cada uno, entendiendo en qué casos son más útiles. Considero que esta actividad me permitió desarrollar un pensamiento más ordenado y lógico para resolver problemas.

Ángel Armando Solís Jiménez 2132088 #55

Este proyecto nos permitió aplicar la teoría sobre la arquitectura y programación de microcontroladores. El mayor reto fue conectar y hacer que los componentes, como botones, LEDs y un display, funcionaran juntos según una lógica predefinida. Investigar las diferencias entre microprocesadores, FPGAs y otros componentes fue clave para entender por qué un microcontrolador era la mejor opción. Además, comparar C y Assembly nos enseñó que C es más productivo, mientras que Assembly ofrece un control más profundo del hardware para optimizar el código.

Durante la realización de esta actividad pude darme cuenta de lo importante que es comprender la lógica detrás de los microcontroladores y cómo se traduce la teoría en resultados prácticos. Al inicio, uno de los mayores desafíos fue organizar la información y entender con claridad cómo aplicar las tablas de verdad en la programación, ya que parecía un tema muy abstracto. Sin embargo, al trabajar con las entradas, salidas y las secuencias de luces, noté cómo todo cobraba sentido y cómo cada detalle en el código influía directamente en el funcionamiento del sistema.

Lo que más valoro de esta experiencia es haber aprendido a combinar el manejo de bits, el control de hardware y la programación en un solo proyecto, lo cual me permitió reforzar mis conocimientos sobre electrónica digital y programación en bajo nivel. Además, enfrentar errores y resolverlos poco a poco me ayudó a desarrollar más paciencia y precisión, cualidades que considero indispensables en esta área.

REFERENCIAS BIBLIOGRÁFICAS

Mauricio, O. (2020). Display 7 segmentos ánodo común | CodigoElectronica. Codigoelectronica.com. <http://codigoelectronica.com/blog/display-7-segmentos-anodo-comun>

Stallings, W. (2017). Computer Organization and Architecture. Pearson.

Mazidi, M., Naimi, S., & Naimi, S. (2018). 8051 Microcontroller and Embedded Systems. Pearson.

Kernighan, B., & Ritchie, D. (1988). The C Programming Language. Prentice Hall.

Maxfield, C. (2004). The Design Warrior's Guide to FPGAs. Elsevier.

Kip Irvine, R. (2015). Assembly Language for x86 Processors. Pearson.

Rabey, J. (2009). Digital Integrated Circuits. Prentice Hall.

Arduino. (2025). Arduino UNO Rev3 with Long Pins. Arduino.cc. <https://docs.arduino.cc/retired/boards/arduino-uno-rev3-with-long-pins/>

Gadgetronicx Team. (2021). Assembly Language – features, uses, advantages and disadvantages. Gadgetronicx. <https://www.gadgetronicx.com/assembly-language-features-uses-advantages-disadvantages>

Marin, D. (2012). What Are Some Advantages/Disadvantages of Using C over Assembly? Software Engineering Stack Exchange. <https://softwareengineering.stackexchange.com/questions/138924/what-are-some-advantages-disadvantages-of-using-c-over-assembly>

Harvie, L. (2024). C vs Assembly: Performance & Readability in Embedded Systems. RunTime Recruitment. <https://runtime-rec.com/c-vs-assembly>